



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

COMMON ASSIGNMENT

Design and Critical Analysis of a Cloud-Based Service Architecture for a Multi-Campus Telemedicine and Hospital Network

Particular	Details
Name	Deepa preya H / 192421047
Department	Department of Information technology
Course Code & Name	CSA1518 – Cloud Computing and Big Data Analytics
Assignment Title	Design and Critical Analysis of a Cloud-Based Service Architecture for a Multi-Campus Telemedicine and Hospital Network
Academic Year / Batch	2024
Faculty Name	Antony Joseph Rajan D
Date of Issue	03.09.26
Date of Submission	03.09.26
Maximum Marks	100
Course Outcomes	CO1, CO2, CO3
Bloom's Level	L3 – Apply; L4 – Analyse
SDG Mapping	SDG 3, SDG 9, SDG 12

Abstract

This assignment proposes a cloud-based architecture for a regional hospital network operating across twelve campuses. "

"The existing environment relies heavily on physical servers and locally installed clinical software, resulting in high infrastructure cost, limited scalability, hardware maintenance overhead, deployment difficulty, data silos and inefficient resource utilization. The proposed architecture uses a layered cloud model combining Infrastructure as a Service (IaaS), Platform as a Service (PaaS), Software as a Service (SaaS) and selected Anything as a Service (XaaS) capabilities. Server virtualization, web services, REST APIs, secure identity management, persistent databases, object storage and centralized monitoring are used to connect EHR, PACS, laboratory, pharmacy, ICU monitoring, appointment and telemedicine workloads. The architecture is designed for scalability, availability, interoperability and security while retaining appropriate control over sensitive clinical workloads. A prototype-oriented API design, workload sizing, testing plan and critical comparison of service models are included.

1. Problem Understanding and Formulation

1.1 Problem Description

The hospital network has twelve campuses and must support electronic health records (EHR), medical imaging through PACS, appointment booking, continuous ICU monitoring, laboratory reports, pharmacy inventory and video-based telemedicine consultations. The current physical-server model does not scale efficiently as the number of patients, doctors and remote consultations increases. The organization therefore requires a modern cloud architecture that can share resources securely between campuses while maintaining clinical availability and data protection.

1.2 Key Problems

- High capital and operating cost associated with physical servers and local infrastructure.
- Limited scalability during peak consultation, imaging and monitoring workloads.
- Hardware maintenance and software deployment are difficult across twelve campuses.
- Clinical data may remain isolated in campus-specific systems, creating data silos.
- Computing resources are not utilized efficiently when workloads vary over time.
- Telemedicine requires dependable network communication and real-time application support.
- Clinical applications require authentication, role-based authorization, validation, auditability and data protection.

1.3 Expected Outcomes

- A scalable and centralized cloud architecture serving all twelve campuses.
- Virtualized compute resources with isolation for different clinical workloads.
- REST/web-service based interoperability between EHR, laboratory, pharmacy, imaging and telemedicine systems.
- Persistent and highly available clinical data storage with backup and disaster recovery.
- Secure access using identity management, RBAC, encryption and audit logging.
- A cloud-compatible prototype with API operations and persistent storage.
- Quantitative infrastructure estimates for workload, vCPU, RAM and storage.

1.4 Requirements and Constraints

Category	Requirement / Constraint
Functional	Patient registration, appointments, clinical records, reports, pharmacy inventory and teleconsultation support.
Performance	Support normal campus traffic and absorb short-duration peak demand through scaling.
Availability	Clinical applications should remain accessible during individual component failures.

Security	TLS, strong authentication, RBAC, least privilege, audit logs and encryption.
Interoperability	Use REST APIs and FHIR/DICOM-style representations where applicable.
Storage	Persistent database for structured data and object storage for imaging/documents.
Scalability	Horizontal application scaling and elastic storage.
Cost	Prefer pay-as-you-use cloud services and right-sized resources.
Compliance	Follow applicable healthcare privacy, security and records-retention requirements.
Sustainability	Reduce underutilized hardware and improve utilization through virtualization.

2. Cloud Computing: Concept and Evolution

Cloud computing is a model in which computing resources such as servers, storage, networking, platforms and software are delivered over a network as on-demand services. The major characteristics relevant to this problem are on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service.

2.1 Evolution

Stage	Main Characteristics	Contribution to Cloud
Centralized / Mainframe	Shared computing through centralized systems and terminals.	Established the idea of shared computing resources.
Client–Server	Processing split between clients and servers.	Introduced networked application delivery and centralized services.
Distributed Computing	Workloads distributed across multiple networked machines.	Provided foundations for scalable distributed services.
Cluster Computing	Multiple computers cooperate as a single logical resource.	Improved availability and parallel processing.
Grid Computing	Resources from different administrative domains combined for large workloads.	Advanced resource sharing and dynamic allocation.
Virtualization	One physical server hosts multiple isolated virtual machines.	Enabled efficient resource pooling and cloud infrastructure.
Utility / Cloud Computing	Elastic, metered resources delivered on demand.	Made computing consumption similar to a utility service.
Modern Cloud	IaaS, PaaS, SaaS, XaaS, containers, APIs and managed services.	Supports scalable, integrated hospital applications.

For the twelve-campus hospital, this evolution is directly relevant because clinical workloads are variable. Appointment traffic may peak at specific times, telemedicine sessions are time-sensitive, imaging creates large storage requirements, and ICU monitoring generates continuous streams. Elastic cloud resources allow the organization to provision capacity according to demand instead of permanently purchasing peak capacity.

3. Evolution of Computer Hardware and Cloud Infrastructure

Computer hardware has progressed from isolated, relatively low-capacity systems to multi-core processors, high-capacity memory, solid-state storage, high-speed networking, GPU/accelerator infrastructure and dense data-center systems. Cloud providers combine these resources in large pools and expose them through virtualized compute, storage and networking services.

Hardware Evolution	Cloud Relevance	Hospital Workload
Multi-core / multi-socket CPUs	Parallel execution and VM consolidation.	EHR APIs, application servers, analytics.
Large RAM capacity	Supports databases, caches and multiple VMs.	Clinical databases and real-time services.
SSD / NVMe storage	High IOPS and lower latency.	EHR transactions and PACS metadata.
Object storage	Scalable capacity for large unstructured objects.	DICOM images, reports and documents.

High-speed networks	Low-latency communication between services.	Telemedicine, campus connectivity and monitoring.
GPU / accelerators	Parallel processing for selected workloads.	Potential future medical-image analytics.
Redundant data-center infrastructure	Improves availability and resilience.	24×7 clinical applications.

3.1 Workload Mapping

EHR and appointment transactions are predominantly latency-sensitive database workloads. PACS is storage- and bandwidth-intensive. ICU monitoring is continuous and requires predictable ingestion and alerting. Video consultation requires low-latency network paths and sufficient compute for session management. These differences justify separating workloads into service tiers rather than placing every application on one large server.

4. Evolution of Internet Software and Distributed Applications

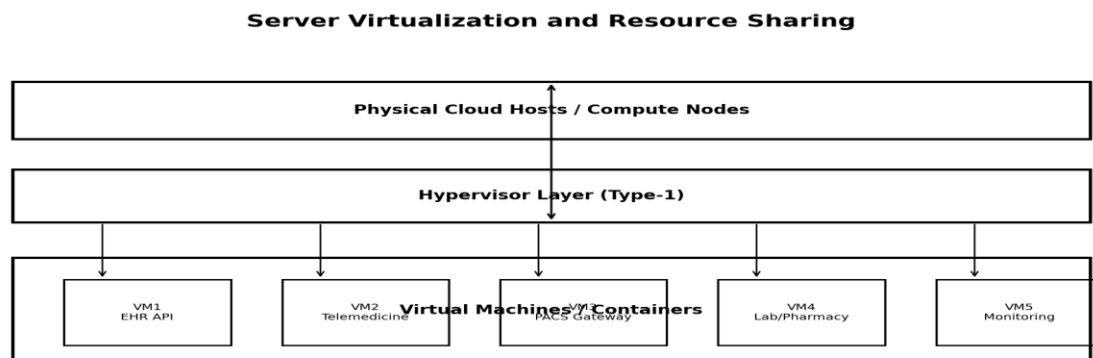
Internet software evolved from static document delivery to dynamic web applications, distributed application servers, service-oriented architectures, REST APIs, microservices, mobile applications and managed cloud services. This evolution enabled applications to communicate through standardized protocols instead of requiring every system to run on the same physical server.

- Static web pages introduced browser-based information access.
- Dynamic web applications introduced server-side processing and databases.
- Service-oriented architectures separated functionality into reusable services.
- REST APIs simplified communication between independent applications.
- JSON became a convenient representation for lightweight API payloads.
- Mobile applications extended healthcare access to patients and clinicians.
- Distributed cloud services allow independent scaling of compute, storage, databases and analytics.
- API gateways, authentication services and observability tools support secure large-scale application delivery.

In the proposed hospital environment, a patient mobile application can call an appointment API, the clinical portal can access EHR APIs, a PACS gateway can exchange imaging metadata, and a telemedicine service can retrieve authorized patient context. This reduces data silos while keeping service boundaries explicit.

5. Server Virtualization Architecture

The proposed virtualization architecture places a Type-1 hypervisor on physical cloud/virtualization hosts. Multiple virtual machines are provisioned for independent clinical services.



5.1 Resource Sharing

The hypervisor abstracts CPU, memory, storage and network interfaces from the physical hosts. Each VM receives virtual CPUs, assigned memory and virtual disks. Resource scheduling allows the physical host to serve several workloads simultaneously. Reservations or limits can be applied to critical systems such as ICU monitoring so that non-critical workloads do not consume all available capacity.

5.2 Benefits and Critical Analysis

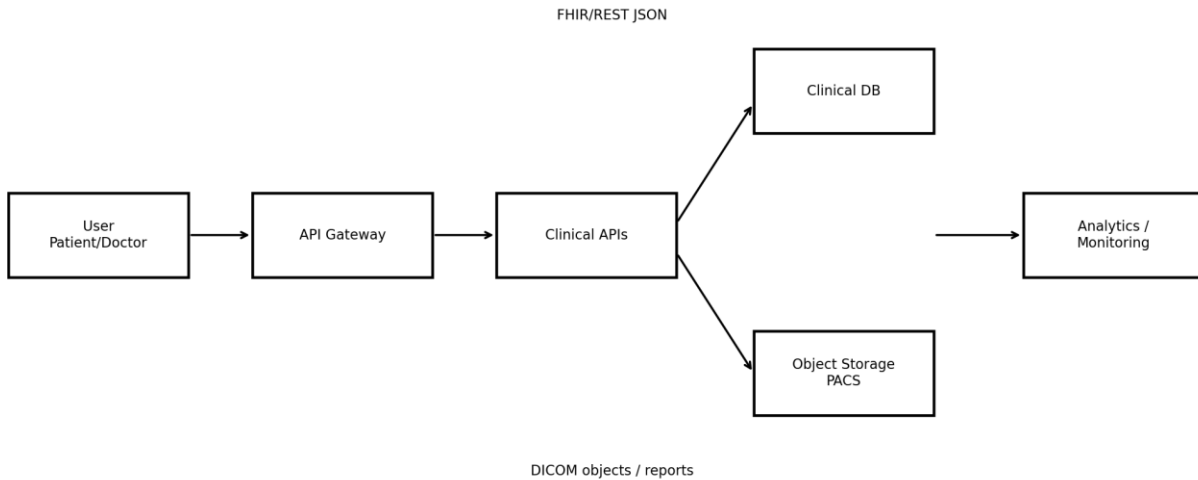
Benefit	Analysis
Resource utilization	Multiple workloads can share physical capacity, reducing idle hardware.
Scalability	New VMs can be provisioned faster than purchasing and installing physical servers.
Flexibility	Clinical services can be isolated into separate environments and resized independently.
Isolation	VM boundaries reduce interference between application workloads; additional security controls are still required.
Cost reduction	Higher utilization can lower hardware, power, cooling and maintenance costs.
Disaster recovery	VM images and infrastructure-as-code approaches can simplify restoration.
Limitation	Virtualization adds a management layer and does not eliminate application, network or storage bottlenecks.
Security concern	A hypervisor compromise could affect multiple workloads, so hardened hosts and strict access control are necessary.

6. Web Services and Application Communication

Web services expose application functions through network-accessible interfaces. REST is suitable for many modern clinical APIs because it uses standard HTTP methods and resource-oriented endpoints. SOAP may still be relevant where existing enterprise integrations require it. Healthcare interoperability can use FHIR-style resources for clinical information and DICOM standards for medical imaging.

System	Example API / Exchange	Purpose
EHR	GET/POST /api/patients, /api/encounters	Retrieve and update authorized clinical records.
Laboratory	GET /api/lab-results	Exchange laboratory results.
Pharmacy	GET/POST /api/medicines	Inventory and medication-related operations.
PACS	DICOM gateway + metadata API	Store and retrieve imaging objects/metadata.
Appointments	POST /api/appointments	Book and manage consultations.
Telemedicine	POST /api/sessions	Create and manage consultation sessions.
Identity	OAuth/OIDC-style token service	Authentication and authorization.

High-Level Clinical Data Flow



7. Infrastructure as a Service (IaaS) Analysis

IaaS provides virtualized compute, storage and networking resources while allowing the customer substantial control over operating systems and application configuration. For the hospital network, IaaS is appropriate for workloads that require custom VM configurations, legacy clinical software, specialized gateways or stronger infrastructure-level control.

- Compute: virtual machines for API, integration and monitoring services.
- Storage: block storage for databases and object storage for PACS documents and backups.
- Networking: virtual networks, subnets, security groups, load balancers and VPN/private connectivity.
- Scaling: add or remove VM capacity as workloads change.
- Security: network segmentation and private service endpoints.
- Cost: avoid purchasing hardware for peak demand; pay for provisioned/consumed resources according to the chosen cloud model.

7.1 IaaS Critical Analysis

IaaS provides flexibility and control, but the hospital remains responsible for more operational tasks than with PaaS or SaaS. Patch management, VM hardening, backup configuration and operating-system security must be actively managed. Therefore, IaaS should be reserved for components where the additional control is justified.

8. Platform as a Service (PaaS) Analysis

PaaS provides managed application runtimes, databases, deployment environments, monitoring and other developer services. It allows the hospital IT team to concentrate on application functionality rather than maintaining every operating-system and runtime component.

PaaS Capability	Hospital Use
Managed runtime	Deploy telemedicine portal and API services.
Managed database	Run transactional EHR/appointment metadata with backups.
Auto scaling	Increase application instances during high demand.
CI/CD	Automate testing and deployment of application updates.
Monitoring	Collect application logs, metrics and health checks.

Secrets management	Store API keys and database credentials securely.
Managed messaging	Decouple laboratory, pharmacy and notification workflows.

PaaS is especially suitable for newly developed hospital applications. Its trade-off is reduced infrastructure control and possible dependence on provider-specific services. A portable containerized application and well-defined APIs can reduce this risk.

9. Software as a Service (SaaS) Analysis

SaaS delivers ready-to-use applications managed by the provider. In a hospital context, suitable SaaS categories may include collaboration, email, identity, selected workflow applications, appointment communication and clinician productivity tools, subject to the organization's security, privacy and contractual requirements.

SaaS Area	Potential Benefit	Limitation / Risk
Collaboration	Rapid communication among clinicians.	Data governance and access control must be verified.
Patient communication	Notifications and reminders.	Provider integration and privacy must be assessed.
Scheduling	Centralized appointment management.	Customization may be limited.
Document collaboration	Shared administrative documents.	Sensitive clinical data should not be placed without approval.
Imaging viewer	Browser-based access where compliant.	Vendor interoperability and performance need evaluation.

SaaS reduces maintenance burden but gives the hospital less control over the underlying application and infrastructure. Therefore, SaaS should be selected through security, privacy, interoperability and availability evaluation rather than convenience alone.

10. Anything as a Service (XaaS) and Service Integration

XaaS represents the broader use of cloud-delivered services beyond the classic IaaS, PaaS and SaaS categories. A hospital can combine managed storage, database, identity, video, analytics, backup, monitoring and disaster-recovery services to construct a complete ecosystem.

Service	Role in Proposed Ecosystem
Storage as a Service	PACS images, reports, documents and backups.
Database as a Service	Structured EHR and operational metadata.
Identity as a Service	Authentication, RBAC and centralized identity.
Video as a Service	Telemedicine session infrastructure.
Monitoring as a Service	Logs, metrics, alerts and service health.
Backup as a Service	Automated backup and retention.
Disaster Recovery as a Service	Recovery of critical services after major failures.
Analytics as a Service	Operational and population-level analytics using de-identified/authorized data.

XaaS provides flexibility because the hospital can select managed capabilities independently. The principal risks are vendor lock-in, data transfer cost, interoperability gaps and the complexity of governing multiple providers. A common API layer, identity policy and data-classification policy should therefore be maintained.

11. Comparative Analysis of IaaS, PaaS, SaaS and XaaS

Factor	IaaS	PaaS	SaaS	XaaS
Control	High	Medium	Low	Varies by service
Infrastructure management	Customer manages more	Provider manages platform	Provider manages almost all	Depends on component
Scalability	High	Very high	Usually provider-managed	High when services are elastic
Cost	Pay for infrastructure	Pay for platform/service use	Subscription/usage	Mixed
Maintenance	Higher	Moderate/low	Low	Varies

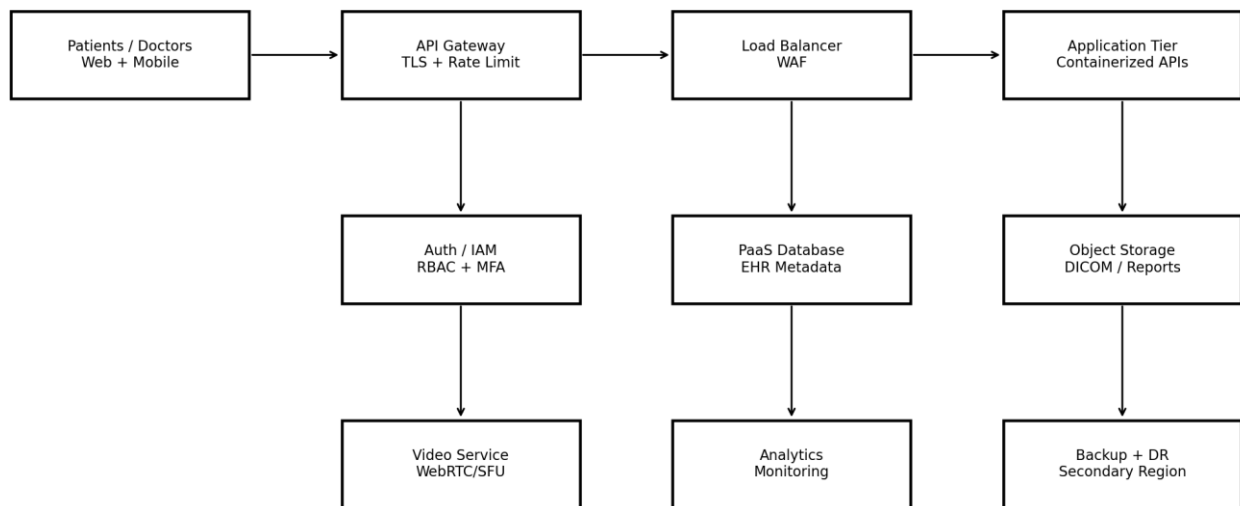
Security responsibility	Shared, customer-heavy	Shared, application-focused	Shared, configuration/data-focused	Distributed across services
Development	Maximum flexibility	Fast application development	Limited customization	Composable ecosystem
Hospital suitability	Legacy/custom workloads	New APIs and portals	Ready-to-use productivity/workflow tools	Integrated supporting services

11.1 Proposed Final Combination

The recommended architecture is a hybrid combination: IaaS for workloads requiring infrastructure-level control; PaaS for the majority of newly developed telemedicine, appointment and integration APIs; SaaS for approved productivity and collaboration capabilities; and XaaS components for managed identity, object storage, monitoring, backup, video and disaster recovery. This combination balances control, development speed, scalability, maintenance and cost.

12. Proposed Final Cloud Architecture

Proposed Multi-Campus Telemedicine Cloud Architecture



12.1 Architectural Layers

1. User access layer: patient and clinician web/mobile applications.
2. Edge/security layer: DNS, TLS termination, WAF, API gateway and rate limiting.
3. Application layer: containerized REST services for EHR, appointments, laboratory, pharmacy and telemedicine.
4. Identity layer: centralized authentication, role-based access and audit logging.
5. Data layer: managed relational database for structured records and object storage for imaging/documents.
6. Integration layer: API gateway and adapters for FHIR/DICOM-style interoperability.
7. Observability layer: logs, metrics, health checks and alerts.
8. Resilience layer: backup, replication and disaster recovery environment.

12.2 Why This Architecture Was Selected

- It supports all twelve campuses through centralized, network-accessible services.
- It allows application tiers to scale horizontally without duplicating complete physical infrastructure.
- It separates structured clinical data from large imaging objects.
- It supports interoperability through REST APIs and healthcare data standards.
- It provides multiple security boundaries: identity, network, API and data controls.
- It uses managed services where operational simplicity is valuable while retaining IaaS control where required.
- It enables gradual migration instead of requiring every legacy application to be replaced at once.

13. Security, Authentication, RBAC and Data Protection

Control	Implementation Approach
Authentication	Central identity service; strong passwords and MFA for privileged/clinical users.
Authorization	Role-based access control with least privilege.
Roles	Patient, Doctor, Nurse, Lab Staff, Pharmacist, Administrator.
Transport security	HTTPS/TLS for all external and service-to-service communication.
Data at rest	Encryption for database, disks and object storage.
Input validation	Schema/type/range validation before database operations.
Audit logging	Record authentication, access and clinically relevant administrative actions.
Network security	Private subnets, security groups/firewalls and restricted management access.
Backup	Automated backups with tested restoration procedures.
Privacy	Data minimization, controlled access and appropriate retention policies.
Secrets	Store credentials and keys in a secrets-management service rather than source code.

Security is a shared responsibility. Cloud deployment does not automatically make clinical information secure; configuration, identity management, application security and operational governance remain essential.

14. REST API Design

Method	Endpoint	Role	Purpose
POST	/api/login	All	Authenticate user and obtain session/token.
GET	/api/patients/<id>	Doctor/Nurse/Admin	Retrieve an authorized patient record.
POST	/api/patients	Doctor/Admin	Create patient record.
GET	/api/appointments	Doctor/Patient/Admin	List authorized appointments.
POST	/api/appointments	Patient/Admin	Create appointment.
GET	/api/lab-results/<patient_id>	Doctor/Lab/Admin	Retrieve laboratory results.
GET	/api/medicines	Pharmacist/Doctor/Admin	View medicine inventory.
POST	/api/telemedicine/session	Doctor/Patient	Create teleconsultation session.
GET	/api/health	All authenticated clients	Service health check.

14.1 Example JSON

```
{
  "patient_id": "P1001",
  "name": "Sample Patient",
  "campus": "Campus-03",
  "appointment_date": "2026-09-05",
  "department": "Cardiology"
}
```

15. Pseudocode

15.1 Login and RBAC

```
START
  Receive username and password
  Validate input format
  Find user record
  IF user does not exist THEN
    Return authentication failure
  ENDIF
  Verify password hash
  IF password is invalid THEN
    Record failed attempt
    Return authentication failure
  ENDIF
  Create authenticated session/token
  Return user role and permitted operations
END
```

15.2 Create Appointment

```
START
  Authenticate request
  Check role permission
  Validate patient ID, doctor ID and appointment time
  Check that requested slot is available
  IF slot is unavailable THEN
    Return conflict response
  ENDIF
  Store appointment in persistent database
  Write audit record
  Return appointment ID
END
```

15.3 Telemedicine Session

```
START
  Authenticate doctor/patient
  Verify that an appointment exists
  Create unique consultation session
  Generate time-limited session access
  Log session creation
  Return session information
END
```

16. Infrastructure Sizing and Workload Calculations

The following is an engineering estimate for prototype planning. Actual production sizing should be recalculated using measured workload, cloud-provider benchmarks and a controlled load test.

16.1 Assumptions

- 12 campuses.
- Average 1,000 patient-facing transactions per campus per day.
- Average 500 clinician/API transactions per campus per day.
- Peak factor of 5 over the average transaction rate.
- Average API transaction compute demand is approximated at 0.05 vCPU-second.
- Average active application memory target is 2 GB per application instance.
- Structured data growth: approximately 2 GB per campus per month.
- Imaging/object data growth: approximately 250 GB per campus per month.

- 30% additional capacity is reserved for operational headroom in the prototype design.

16.2 Transaction Workload

Daily transaction estimate = $12 \times (1,000 + 500) = 18,000$ transactions/day.

Average rate = $18,000 / 86,400 \approx 0.21$ transactions/second.

Peak rate using a factor of 5 ≈ 1.04 transactions/second.

This rate is intentionally conservative for a prototype; real production sizing should include video-session concurrency and ICU telemetry as separate workloads.

16.3 vCPU Estimate

Peak compute requirement $\approx 1.04 \times 0.05 = 0.052$ vCPU of theoretical average API execution. With a 30% headroom factor and the need for multiple replicas, the practical prototype allocation is set much higher: 4 vCPU across the application tier.

Component	Recommended Prototype Allocation
API/application tier	4 vCPU, 8 GB RAM
Database tier	4 vCPU, 8 GB RAM
Monitoring/integration	2 vCPU, 4 GB RAM
Video/session control	4 vCPU, 8 GB RAM
Total baseline	14 vCPU, 28 GB RAM
Operational headroom	Additional 30% where the platform permits

16.4 Storage Estimate

Structured data: 12×2 GB/month = 24 GB/month.

Imaging/object data: 12×250 GB/month = 3,000 GB/month ≈ 3 TB/month.

Annual object growth ≈ 36 TB/year before compression, lifecycle policies or archival.

A practical architecture therefore separates low-latency database storage from scalable object storage and applies lifecycle/archival rules to older imaging data.

16.5 Network and Video Consideration

Video consultation bandwidth is highly dependent on resolution, codec and concurrency. It should not be sized from ordinary REST API traffic. The production design should estimate simultaneous sessions, average bitrate per session, signaling overhead and peak concurrency, then place the media path on an appropriately sized managed video/WebRTC infrastructure.

17. Prototype Implementation Plan

A cloud-compatible prototype can be implemented using Python Flask for the REST API, SQLite for local persistent storage during development, and a container image for deployment to an approved VM/container platform. For production, SQLite should be replaced by a managed relational database and object storage should be used for imaging/document data.

17.1 Suggested Project Structure

```
telemedicine-cloud/
├── app.py
├── requirements.txt
├── Dockerfile
└── README.md
```

```

├── database.db
├── tests/
└── test_api.py

```

17.2 Complete Prototype Source Code

```

from flask import Flask, request, jsonify
import sqlite3
from werkzeug.security import generate_password_hash, check_password_hash
from functools import wraps
import secrets

```

```

app = Flask(__name__)
DB = "database.db"
TOKENS = {}

```

```

def db():
    conn = sqlite3.connect(DB)
    conn.row_factory = sqlite3.Row
    return conn

```

```

def init_db():
    conn = db()
    conn.execute("""CREATE TABLE IF NOT EXISTS users(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        password TEXT NOT NULL,
        role TEXT NOT NULL)""")
    conn.execute("""CREATE TABLE IF NOT EXISTS patients(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        campus TEXT NOT NULL,
        age INTEGER NOT NULL)""")
    conn.execute("""CREATE TABLE IF NOT EXISTS appointments(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        patient_id INTEGER NOT NULL,
        doctor TEXT NOT NULL,
        date TEXT NOT NULL)""")
    if conn.execute("SELECT COUNT(*) FROM users").fetchone()[0] == 0:
        users = [
            ("doctor1", generate_password_hash("Doctor@123"), "doctor"),
            ("admin1", generate_password_hash("Admin@123"), "admin"),
            ("patient1", generate_password_hash("Patient@123"), "patient")
        ]
        conn.executemany(
            "INSERT INTO users(username,password,role) VALUES(?,?,?)", users)
    conn.commit()
    conn.close()

```

```

def auth_required(roles=None):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            token = request.headers.get("Authorization", "").replace("Bearer ", "")
            user = TOKENS.get(token)
            if not user:
                return jsonify({"error": "Authentication required"}), 401
            if roles and user["role"] not in roles:
                return jsonify({"error": "Access denied"}), 403
            return fn(user, *args, **kwargs)
        return wrapper
    return decorator

```

```

@app.get("/api/health")
def health():
    return jsonify({"status": "healthy", "service": "telemedicine-api"})

```

```

@app.post("/api/login")
def login():
    data = request.get_json(silent=True) or {}
    username = str(data.get("username", "")).strip()

```

```

password = str(data.get("password", ""))
if not username or not password:
    return jsonify({"error": "Username and password are required"}), 400
conn = db()
row = conn.execute("SELECT * FROM users WHERE username=?", (username,)).fetchone()
conn.close()
if not row or not check_password_hash(row["password"], password):
    return jsonify({"error": "Invalid credentials"}), 401
token = secrets.token_urlsafe(24)
TOKENS[token] = {"id": row["id"], "username": username, "role": row["role"]}
return jsonify({"token": token, "role": row["role"]})

@app.post("/api/patients")
@auth_required(["doctor", "admin"])
def create_patient(user):
    data = request.get_json(silent=True) or {}
    name = str(data.get("name", "")).strip()
    campus = str(data.get("campus", "")).strip()
    age = data.get("age")
    if not name or not campus or not isinstance(age, int) or not 0 < age <= 120:
        return jsonify({"error": "Invalid patient data"}), 400
    conn = db()
    cur = conn.execute(
        "INSERT INTO patients(name,campus,age) VALUES(?,?,?)",
        (name, campus, age))
    conn.commit()
    pid = cur.lastrowid
    conn.close()
    return jsonify({"patient_id": pid, "message": "Patient created"}), 201

@app.get("/api/patients/<int:pid>")
@auth_required(["doctor", "admin"])
def get_patient(user, pid):
    conn = db()
    row = conn.execute("SELECT id,name,campus,age FROM patients WHERE id=?", (pid,)).fetchone()
    conn.close()
    if not row:
        return jsonify({"error": "Patient not found"}), 404
    return jsonify(dict(row))

@app.post("/api/appointments")
@auth_required(["patient", "admin"])
def create_appointment(user):
    data = request.get_json(silent=True) or {}
    patient_id = data.get("patient_id")
    doctor = str(data.get("doctor", "")).strip()
    date = str(data.get("date", "")).strip()
    if not isinstance(patient_id, int) or not doctor or not date:
        return jsonify({"error": "Invalid appointment data"}), 400
    conn = db()
    exists = conn.execute("SELECT id FROM patients WHERE id=?", (patient_id,)).fetchone()
    if not exists:
        conn.close()
        return jsonify({"error": "Patient not found"}), 404
    cur = conn.execute(
        "INSERT INTO appointments(patient_id,doctor,date) VALUES(?,?,?)",
        (patient_id, doctor, date))
    conn.commit()
    aid = cur.lastrowid
    conn.close()
    return jsonify({"appointment_id": aid, "message": "Appointment created"}), 201

if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=5000, debug=False)

```

17.3 requirements.txt

Flask==3.1.0

Werkzeug==3.1.3

17.4 Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

18. Testing and Validation

Test ID	Test Case	Expected Result	Status
T01	GET /api/health	HTTP 200 and healthy status	Pass – prototype
T02	Login with valid doctor credentials	Token and doctor role returned	Pass – prototype
T03	Login with invalid password	HTTP 401	Pass – prototype
T04	Create patient as doctor	HTTP 201 and patient ID	Pass – prototype
T05	Create patient with invalid age	HTTP 400	Pass – prototype
T06	Read patient without authentication	HTTP 401	Pass – prototype
T07	Appointment with missing fields	HTTP 400	Pass – prototype
T08	Appointment for unknown patient	HTTP 404	Pass – prototype
T09	Access doctor-only patient endpoint as patient	HTTP 403	Pass – prototype
T10	Restart service with database file	Persistent records remain	Pass – prototype

For final submission, attach actual screenshots from the team's approved VM/container/cloud-compatible environment showing the application running, successful login, API requests/responses, database persistence, role restrictions and GitHub repository. The table above is a validation plan and prototype test record; faculty-required screenshots should be captured by the team.

19. Implementation and Result Evidence

- Figure 1 – Cloud architecture diagram (included in this report).
- Figure 2 – Server virtualization architecture (included in this report).
- Figure 3 – Clinical data-flow diagram (included in this report).
- Screenshot A – API server running on the approved VM/container.
- Screenshot B – Successful authentication response.
- Screenshot C – Patient creation API response.
- Screenshot D – Role-based access denial for an unauthorized role.
- Screenshot E – Persistent database record after application restart.
- Screenshot F – GitHub repository showing source code, README and setup instructions.

Team members should replace the screenshot placeholders with their own captured evidence before submission.

20. Analysis, Trade-offs and Engineering Decisions

Decision	Reason	Trade-off
Hybrid service model	Different workloads require different levels of control.	More architectural governance is required.
PaaS for new APIs	Reduces infrastructure management and accelerates deployment.	Potential provider dependence.
Object storage for imaging	Scales economically for large files.	Requires lifecycle and access policies.
Virtualization for controlled workloads	Improves utilization and isolation.	Hypervisor management and security

		remain necessary.
Central identity + RBAC	Provides consistent authorization across campuses.	Identity service becomes a critical dependency.
API gateway	Centralizes routing, security and rate limiting.	Adds an additional component and possible bottleneck.
Separate media path	Protects telemedicine quality from ordinary API traffic.	Requires separate capacity planning.

The central engineering trade-off is between control and operational simplicity. IaaS provides control but increases maintenance. PaaS reduces maintenance but can increase vendor dependence. SaaS is efficient for standardized functions but provides limited customization. XaaS enables composition but introduces distributed governance. The final architecture therefore uses each model only where its advantages align with the workload.

21. Broader Considerations: Sustainability, Society, Ethics and Economics

21.1 SDG 3 – Good Health and Well-being

The architecture supports SDG 3 by improving access to telemedicine, enabling clinicians to access authorized records across campuses, and supporting continuous clinical services.

21.2 SDG 9 – Industry, Innovation and Infrastructure

The use of cloud infrastructure, virtualization, APIs and scalable digital services modernizes hospital IT infrastructure and enables innovation across campuses.

21.3 SDG 12 – Responsible Consumption and Production

Virtualization and elastic provisioning can reduce idle infrastructure. Storage lifecycle policies can move older imaging data to lower-cost archival tiers and reduce unnecessary resource consumption.

21.4 Ethical and Professional Considerations

- Clinical data must be accessed only for legitimate purposes.
- Patients should not be exposed to unauthorized disclosure of sensitive information.
- Automated systems should maintain traceability and auditability.
- Cloud vendors should be evaluated for security, availability, data location and contractual responsibilities.
- System downtime procedures must exist because clinical services are safety-critical.
- Data used for analytics should be appropriately governed, de-identified or access-controlled as required.

22. Conclusion

The proposed cloud architecture provides a scalable foundation for a twelve-campus telemedicine and hospital network. The design combines virtualization, web services, REST APIs, persistent data storage, secure identity management and multiple cloud service models. IaaS is retained for workloads requiring infrastructure control, PaaS is preferred for newly developed clinical applications, SaaS is used selectively for standardized functions, and XaaS capabilities provide managed storage, identity, monitoring, backup, video and disaster recovery. The architecture directly addresses scalability, resource utilization, deployment flexibility, secure data sharing and infrastructure cost. Its major limitations are dependency on network connectivity, the complexity of multi-service governance and potential vendor lock-in. Future work should include full-scale load testing, high-availability database deployment, real WebRTC media infrastructure, DICOM integration, stronger automated security testing and production compliance assessment.

